

DESENVOLVIMENTO DE SISTEMAS WEB I

Aula 1: Introdução ao Desenvolvimento Web com JSP

Da internet até o servidor: como uma página chega até o navegador

Duração: 2 horas

Objetivos da aula

1

Explicar o caminho de uma requisição

Do navegador até o servidor, e de volta.

2

Diferenciar cliente e servidor

E entender onde o JSP entra nesse modelo.

3

Entender o papel do DNS

E como um domínio se conecta a um servidor real.

4

Escrever HTML estrutural e semântico

Incluindo um formulário funcional.

5

Diferenciar validação client-side e server-side

E por que as duas são necessárias.

Como a internet funciona



IP

Endereço único de cada dispositivo na rede.



Pacotes

Os dados são divididos e podem seguir rotas diferentes.



Roteadores

Decidem o caminho de cada pacote até o destino.



TCP/IP

IP endereça e roteia; TCP garante entrega confiável (ACK + timeout).

ANALOGIA EM SALA

A internet como sistema de correios global

Sua casa tem um endereço (IP). Uma carta grande é dividida em envelopes menores (pacotes), que podem seguir rotas diferentes e são remontados na ordem certa no destino.

Pergunta para a turma: por que dividir os dados em pacotes, em vez de enviar tudo de uma vez?

HTTP: requisição e resposta

REQUISIÇÃO (cliente)

- Método: GET, POST, PUT, DELETE
- URL do recurso
- Headers (metadados)
- Body (dados enviados, se houver)

HTTP é stateless: cada requisição é independente do servidor.

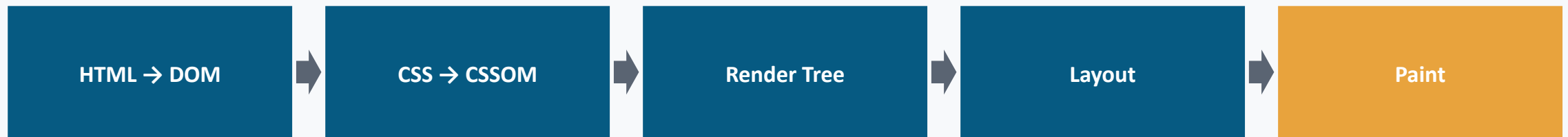


RESPOSTA (servidor)

- Status code: 200, 404, 500...
- Headers (tipo do conteúdo, etc.)
- Body (HTML, JSON, imagem...)

2xx sucesso · 3xx redirecionamento · 4xx erro do cliente · 5xx erro do servidor

Como o navegador renderiza a página

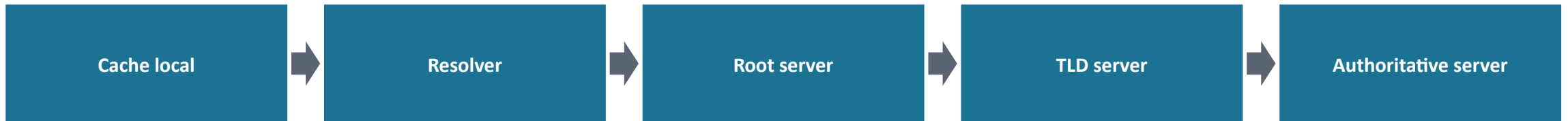


O JavaScript pode alterar o DOM/CSSOM em execução, reiniciando o ciclo Layout → Paint.

CONEXÃO COM JSP

O HTML gerado dinamicamente pelo servidor (JSP) passa exatamente por esse mesmo pipeline no navegador do usuário — DOM, CSSOM, Layout e Paint não sabem se o HTML veio de um arquivo estático ou foi montado na hora pelo servidor.

DNS: a jornada de uma consulta



Resultado final: o IP do servidor onde o site está hospedado.

ANALOGIA EM SALA

Pedir informação na recepção de um prédio enorme: a portaria (resolver) te encaminha ao andar certo (TLD server), que te encaminha à recepção daquele andar (authoritative server), que finalmente diz "sala 305" (o IP).

Anatomia de um domínio e onde ele mora

DOMÍNIO

`www.exemplo.com.br`

- Subdomínio (www) — seção dentro do domínio
- SLD (exemplo) — a marca/identidade
- TLD (.com.br) — domínio de topo

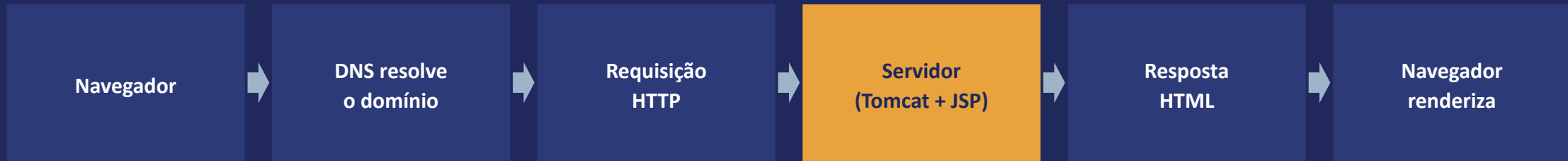
Domínio ≠ hospedagem: o domínio é só o nome.

HOSPEDAGEM

- Compartilhada — barata, recursos divididos
- VPS — comum para apps Java com Tomcat
- Dedicado — máquina inteira só para a aplicação
- Nuvem (AWS, GCP, Azure) — escalável sob demanda

Para rodar JSP, o servidor precisa de um Servlet Container (Apache Tomcat) — é ele quem processa o JSP e devolve o HTML.

Do navegador ao servidor, e de volta



O JSP é código Java que roda dentro do Tomcat e gera o HTML dinamicamente antes de devolver a resposta.

Estrutura e semântica

ESQUELETO DO DOCUMENTO

```
<!DOCTYPE html>
<html>
  <head>
    <title>...</title>
  </head>
  <body>
    ...
  </body>
</html>
```

head = metadados (não aparece na tela) · body = conteúdo visível

TAGS SEMÂNTICAS

- header — cabeçalho da página/seção
- nav — bloco de navegação
- main — conteúdo principal (só um por página)
- article — conteúdo independente e autocontido
- section — agrupamento temático
- footer — rodapé

Por quê: acessibilidade, SEO e manutenibilidade do código.

Formulários: a porta de entrada de dados

```
<form action="/cadastro" method="POST">
  <label for="nome">Nome:</label>
  <input id="nome" name="nome" required>
  <button type="submit">Enviar</button>
</form>
```

A PONTE COM O JAVA

O atributo name="nome" do HTML vira exatamente a string usada no servidor:

`request.getParameter("nome")`

action define o Servlet de destino · method define GET ou POST

VALIDAÇÃO CLIENT-SIDE vs SERVER-SIDE

Client-side (required, type) dá feedback rápido, mas é contornável. Server-side é obrigatória — nunca se confia 100% no que vem do cliente.

Onde o JSP entra nessa história

HTML estático vs. gerado dinamicamente

Estático

O mesmo arquivo HTML é sempre devolvido, sem mudar.

Dinâmico (JSP)

O servidor monta o HTML na hora, com dados de um formulário, de um banco, etc.

PRÉVIA: PADRÃO MVC (próxima aula)

Model (dados) · View (JSP gerando HTML) · Controller (Servlet recebendo a requisição)

O <form> que vocês aprenderam hoje é exatamente o que, na próxima aula, vai enviar dados para um Servlet processar.

Cadastro de aluno em HTML

Monte uma página com estrutura semântica e um formulário funcional:

- header com um título e footer com seu nome
- main envolvendo todo o conteúdo principal
- form com action="/cadastro-aluno" e method="POST"
- Campo nome (name="nome", required)
- Campo email (type="email", name="email", required)
- Campo de data de nascimento (type="date")
- Cada campo com label associado (for + id)
- Botão de envio (type="submit")

Ao final: para onde iriam esses dados se o formulário fosse enviado agora? Quem processaria?

RESUMO

Hoje vimos como uma página chega até o navegador.

Na próxima aula, vamos abrir o lado do servidor e ver como o JSP gera essa página — estrutura MVC aplicada ao desenvolvimento web.

Próxima aula: Estrutura MVC (Model-View-Controller)